

Competitive Programming

Section 36: Fenwick Trees

Competitive Programming Course

2026-07-06

Table of contents

1	Fenwick trees without black boxes	3
1.1	Building intuition with one update	4
2	Coordinate compression with Fenwick	5
3	Difference arrays and Fenwick	6
4	Checklist	7
5	Minimal templates	8
6	Local exercises	9
7	How to use the checker	10

1 Fenwick trees without black boxes

A Fenwick tree, also called a binary indexed tree, stores prefix information in an array `bit[1..n]`. The key idea is that index `i` owns a block whose length is the value of its lowest set bit:

```
lowbit(i) = i & -i

bit[i] stores the sum of:
i-lowbit(i)+1, ..., i
```

Example for `n = 8`:

i	lowbit(i)	covered segment
1	1	[1,1]
2	2	[1,2]
3	1	[3,3]
4	4	[1,4]
5	1	[5,5]
6	2	[5,6]
7	1	[7,7]
8	8	[1,8]

To compute a prefix sum `sum(i)`, repeatedly take the block ending at `i`, then jump to the index just before that block:

```
answer += bit[i]
i -= i & -i
```

This decomposes `[1,i]` into disjoint power-of-two blocks. For `i = 13`, the visited indices are:

```
13 -> 12 -> 8 -> 0
[13,13] + [9,12] + [1,8]
```

To add `delta` to position `i`, every Fenwick block containing `i` must increase by `delta`. Those block owners are found by moving upward:

```
bit[i] += delta
i += i & -i
```

Both loops touch at most one index per binary digit, so each operation costs $O(\log n)$.

1.1 Building intuition with one update

Suppose $n = 8$ and we add $+5$ at position 3.

The update visits:

```
3 -> 4 -> 8
```

Those are exactly the Fenwick blocks that contain position 3:

```
bit[3] covers [3,3]
bit[4] covers [1,4]
bit[8] covers [1,8]
```

No other block ending at another index includes position 3, so no other Fenwick entry changes.

For $\text{sum}(6)$, the query visits:

```
6 -> 4 -> 0
```

and reads:

```
bit[6] covers [5,6]
bit[4] covers [1,4]
```

Together these blocks form $[1,6]$ without overlap.

2 Coordinate compression with Fenwick

Fenwick indices must be dense integers. If values are as large as 10^9 , sort the distinct values and replace each value by its 1-based rank. Equal values receive the same rank. This keeps order:

```
x < y  iff  rank(x) < rank(y)
```

In inversion counting, when reading `a[i]`, the number of previous elements greater than it is:

```
previous_count - count(previous values <= a[i])
```

A Fenwick tree over ranks gives that count.

3 Difference arrays and Fenwick

A normal difference array has:

```
diff[1] = a[1]
diff[i] = a[i] - a[i-1]
a[i] = diff[1] + ... + diff[i]
```

Adding x to a range $[l, r]$ changes only two difference entries:

```
diff[l] += x
diff[r+1] -= x
```

If a Fenwick tree stores `diff`, range-add and point-query both become logarithmic.

4 Checklist

```
Use 1-based indices internally.  
Decide whether the tree stores values or differences.  
For range sums, answer  $\text{sum}(r) - \text{sum}(l-1)$ .  
For inversions, count strictly greater values; duplicates are not inversions.  
Use 64-bit integers for counts and sums.
```

5 Minimal templates

C++:

```
struct Fenwick {
    int n;
    vector<long long> bit;
    Fenwick(int n) : n(n), bit(n + 1, 0) {}
    void add(int i, long long delta) {
        for (; i <= n; i += i & -i) bit[i] += delta;
    }
    long long sum(int i) const {
        long long ans = 0;
        for (; i > 0; i -= i & -i) ans += bit[i];
        return ans;
    }
    long long range_sum(int l, int r) const {
        return sum(r) - sum(l - 1);
    }
};
```

Python:

```
class Fenwick:
    def __init__(self, n):
        self.n = n
        self.bit = [0] * (n + 1)

    def add(self, i, delta):
        while i <= self.n:
            self.bit[i] += delta
            i += i & -i

    def sum(self, i):
        ans = 0
        while i > 0:
            ans += self.bit[i]
            i -= i & -i
        return ans
```

6 Local exercises

- A. Point Add Range Sum: Maintain an array. Operation 1 i x adds x to $a[i]$. Operation 2 l r asks for the sum of $a[l..r]$.
- B. Inversion Count: Given an array, count pairs $i < j$ with $a[i] > a[j]$. Values may repeat and may be large.
- C. Range Add Point Query: Maintain an array. Operation 1 l r x adds x to every element in $[l,r]$. Operation 2 i asks for the current value of $a[i]$.

7 How to use the checker

The local checker compiles/runs both languages and compares output tokens exactly:

```
CP_TARGET=solution python3 sections/36_fenwick_trees/check.py
```

Replace `CP_TARGET=solution` by your own `solve.cpp` or `solve.py` attempts when practicing.